# 8-puzzle A* implementation

## Code:

```
import heapq

def misplaced_tiles(state, goal):
    return sum(state[i] != goal[i] and state[i] != 0 for i in range(9))

def manhattan_distance(state, goal):
    distance = 0
    for i in range(9):
        if state[i] != 0:
            current_row, current_col = divmod(i, 3)
            goal_pos = goal.index(state[i])
            goal_row, goal_col = divmod(goal_pos, 3)
            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)
    return distance

def get_successors(state):
    successors = []
    blank = state.index(0)
    row, col = divmod(blank, 3)


    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col
            new_state = list(state)
            new_state[blank], new_state[new_blank] = new_state[new_blank], new_state[blank]
            successors.append(tuple(new_state))
    return successors

def a_star(initial, goal, heuristic):
    open_list = []
    heapq.heappush(open_list, (0, initial, 0, [initial]))
    closed = set()
    while open_list:
        f, state, g, path = heapq.heappop(open_list)
        if state == goal:
            return path
        if state in closed:
            continue
        closed.add(state)
        for successor in get_successors(state):
```

```
            if successor not in closed:
                new_g = g + 1
                h = heuristic(successor, goal)
                new_f = new_g + h
                heapq.heappush(
                    open_list, (new_f, successor, new_g, path + [successor])
                )
    return None

initial = (5, 4, 6, 1, 0, 8, 7, 3, 2)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

path = a_star(initial, goal, misplaced_tiles)

if path:
    for state in path:
        print(state[0:3])
        print(state[3:6])
        print(state[6:9])
        print()
else:
    print("No solution found.")
```

## Output:

```
(5, 4, 6)
(1, 0, 8)
(7, 3, 2)

(5, 0, 6)
(1, 4, 8)
(7, 3, 2)

(0, 5, 6)
(1, 4, 8)
(7, 3, 2)

(1, 5, 6)
(0, 4, 8)
(7, 3, 2)

(1, 5, 6)
(4, 0, 8)
(7, 3, 2)

(1, 5, 6)
(4, 3, 8)
(7, 0, 2)

(1, 5, 6)
(4, 3, 8)
(7, 2, 0)

(1, 5, 6)
(4, 3, 0)
(7, 2, 8)

(1, 5, 0)
(4, 3, 6)
(7, 2, 8)

(1, 0, 5)
(4, 3, 6)
(7, 2, 8)

(1, 3, 5)
(4, 0, 6)
(7, 2, 8)

(1, 3, 5)
(4, 2, 6)
(7, 0, 8)

(1, 3, 5)
(4, 2, 6)
(7, 8, 0)

(1, 3, 5)
(4, 2, 0)
(7, 8, 6)
```

```
(1, 3, 5)
(4, 2, 6)
(7, 0, 8)

(1, 3, 5)
(4, 2, 6)
(7, 8, 0)

(1, 3, 5)
(4, 2, 0)
(7, 8, 6)

(1, 3, 0)
(4, 2, 5)
(7, 8, 6)

(1, 0, 3)
(4, 2, 5)
(7, 8, 6)

(1, 2, 3)
(4, 0, 5)
(7, 8, 6)

(1, 2, 3)
(4, 5, 0)
(7, 8, 6)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```